Context Integrity in LLM Agents

Two research communities study the same failure. Neither tests the other's work.
We measured what that costs.

Nyamtulla · Advisor meeting · 7 October 2026

An agent believes everything in its context window

An LLM agent reads one block of text and then acts. We call this block the context window.

The block holds the user's question. It also holds a page a tool returned, a document a search step found, and a note the agent stored last week.

The model cannot tell these apart. Everything in the block looks the same to it.

So any text that reaches the block can change what the agent does. The text does not need to come from the user. It only needs to arrive.

This is not about training data. It is about what enters the context while the agent runs.

Two communities. One failure. No contact.

The security literature

It treats the text as an attack. Someone wrote it to take the agent over.

Indirect prompt injection. Poisoned retrieval. Poisoned memory.

The machine learning literature

It treats the same corruption as an accident. Nobody wrote it to cause harm.

Lost in the middle. Distraction by irrelevant text. Knowledge conflict.

Both build defenses. Each side tests its defenses on its own failures.

A tool returns a page with a hidden order. A tool returns a page that is ten times too long. Both arrive through the same route. Both end in the same window. One has an author and one does not, and that is the only difference.

Our question: does that difference justify two separate sets of defenses?

Security end: Greshake et al., AISec@CCS 2023, arXiv:2302.12173 · ML end: Liu et al., TACL 2023, arXiv:2307.03172; Shi et al., ICML 2023, arXiv:2302.00093 · Knowledge conflict: Xu et al., EMNLP 2024, arXiv:2403.08319

We describe every case with three questions

Channel	How did the text get in? Which route did it travel?
Intent	Did a person put it there on purpose, or not?
Consequence	What went wrong once the text was in the window?

9 channels × 3 intents × 6 consequences = 162 combinations.

A combination with no paper in it is a question the field has never asked. We find that 98 of the 162 are empty.

The framework is the spine of the work. It tells us what is known, and it tells us where to look next.

CHANNEL

Nine routes carry text into the window

tool-output	What a tool gives back: a web page, an email, a file the agent opened.	**75**
direct-input	The prompt itself. The user's text, or the system prompt above it.	**67**
RAG	A document a search step pulled out of a corpus.	**28**
cross-modal	Text carried inside an image, audio or video.	**18**
memory	Something the agent stored earlier and now reads again.	**16**
tool-metadata	The description of a tool, not its output.	**13**
multi-agent	A message from another agent in the same system.	**10**
supply-chain	A package or template the agent inherits before it starts.	**6**
skill	A stored procedure the agent loads and follows.	**4**

The right column counts the named attacks in that channel.

tool-output Greshake et al., AISec@CCS 2023 · direct-input Liu et al., 2023 · RAG Zou et al., USENIX Security 2024 · memory Chen et al., NeurIPS 2024 · cross-modal Wang et al., EMNLP 2025
tool-metadata Wang et al., AAAI 2026 · multi-agent Lee et al., ESORICS 2025 · supply-chain Qu et al., 2026 · skill Jia et al., 2026

CONSEQUENCE

Six ways the agent then fails

goal-hijack	The agent does another person's task. Text in the window gives an order and it obeys.	**136**
reasoning-corruption	Every word is true. The window is too long or too cluttered, and the model stops using it well.	**46**
silent-corruption	The agent answers from false content. Nobody gave it a new task. It believes a wrong thing.	**21**
persistence-backdoor	The bad content is stored. It fires on a later run, maybe for another user.	**16**
data-exfiltration	The agent sends out what it can read, through a tool call, a link or a message.	**14**
resource-abuse	No data moves and no goal changes. The agent burns time, or refuses to answer.	**4**

The second row is the machine learning literature. The other five are the security literature. That is the split, in one column.

goal-hijack Greshake et al., AISec@CCS 2023 · reasoning-corruption Liu et al., TACL 2023; Shi et al., ICML 2023 · silent-corruption Xie et al., ICLR 2023 · persistence-backdoor Chen et al., NeurIPS 2024 · data-exfiltration Qi et al., ICLR 2024 · resource-abuse Xue et al., 2024

A map of every attack and every defense in the field

1,026 papers	read in full and coded, drawn from about 26,000 candidates
237 attacks	distinct named ways to corrupt a context window
534 defenses	distinct named ways the field protects against them
349 pairs	every case where a paper tested one defense against one attack

No prior survey holds both literatures. The closest one reviews 78 papers, 37 attacks and 41 defenses, and stays on the security side.

That map is the instrument. Everything that follows is read off it.

FINDING 1

The two communities do not read each other

11.7 %	of security papers cite at least one machine learning paper
9.1 %	of machine learning papers cite at least one security paper
98.0 %	of all tested pairs stay inside one literature
60.5 %	of the 162 combinations in the framework have never been studied

Nine of ten papers never cross. Seven pairs in the whole field do.

This is not a sample. We resolved every citation edge in the corpus.

So every claim of the form “this defense is robust” holds inside one threat model. Nobody has checked the other one.

FINDING 2

Most defenses have never met most attacks

119 of 237 attacks have no tested defense at all

15 of 534 defenses were tested against both threat models. That is 2.8 %.

A defense paper is not a neutral document. A defense is supposed to defend something.

99 defense papers name an attack that sits in our map, and then never test against it. Those are not gaps in the field's knowledge. They are gaps in its evidence.

At this point the survey had taken us as far as a survey can go. It can show that a test was never run. It cannot say what the test would have found.

We ran the tests the field never ran

We took defenses with released code, and attacks with released code, and ran the pairs that no paper reports.

18 defense and attack pairs. 104 measured conditions. Four attacks. Five defenses. Three models.

We raise the attack dose from zero to ten poisoned passages out of ten, so each pair gives a curve and not a single point.

Two rules decide what we are allowed to claim.

The control first	A defense must show its own benefit on its own task before we read anything else from it.
A size floor	We fix the smallest change the sample can detect. Below that we report nothing, in either direction.

Scoring comes from each paper's own code, unchanged.

RESULT 1

One family of defenses fails under pressure. The other does not.

When every retrieved passage belongs to the attacker:

Context-reliance	It holds down the model's own memory so the model follows the retrieved text.	**4 of 5**
Isolate-then-aggregate	It answers from each passage alone, then joins the answers.	**0 of 10**

readable cases that lose accuracy

The loss is the same size every time: about 32 points, on two defenses, two attacks and two models.

Against a false passage the loss begins at eight poisoned passages of ten. It does not wait for all ten.

The design rule that follows: do not suppress the model's own memory. It is the only thing left when the evidence runs out.

ParamMute Huang et al., NeurIPS 2025 · RobustRAG Xiang et al., 2024 · PoisonedRAG Zou et al., USENIX Security 2024 · BadRAG Xue et al., 2024 · Measurements: this work

RESULT 2

Some defenses make an attack work that otherwise does nothing

Defense	Attack	No defense	With the defense	Poisoned
CK-PLUG	tone steering	0.0 %	**78.2 %**	1 of 10
CK-PLUG	tone steering	4.0 %	**99.0 %**	10 of 10
ParamMute	tone steering	3.4 %	**28.8 %**	10 of 10

With no defense the attack stays at or near zero. It only works once the defense is switched on.

One poisoned passage out of ten is enough. The undefended model ignores it completely.

The reason is uncomfortable. Tone steering does not change the answer, so a defense that pushes the model to follow the retrieved text follows the slant as well, and scores itself as working perfectly. The attack and the defense want the same thing.

A defense can be the reason an attack succeeds. That is a security finding, not a performance note.

BadRAG Xue et al., 2024 · ParamMute Huang et al., NeurIPS 2025 · Measurements: this work

RESULT 3

One defense, two models, opposite results

One defense family, two versions of it, two models. Nine poisoned passages of ten. The number is how often the agent refuses to answer.

Model	Version of the defense	No defense	With the defense
Mistral-7B	KeywordAgg	0.0 %	**5.7 %**
Mistral-7B	DecodingAgg	0.0 %	**3.7 %**
Llama-3-8B	KeywordAgg	0.0 %	**85.0 %**
Llama-3-8B	DecodingAgg	0.0 %	**85.3 %**

The undefended model never refuses. Not once, on either model, at any dose.

The cause is the defense's own design. It answers from each passage alone, so a passage that says “refuse” gets an unopposed hearing. The undefended model reads all ten passages together, where nine honest ones drown it out.

It is the same property that makes this family resist tone steering. One trade, not two results, and the model decides which way it falls.

RobustRAG, both versions Xiang et al., 2024 · BadRAG Xue et al., 2024 · Measurements: this work

A survey found the gap. Running the code showed what is in it.

It is new evidence	16 defense and attack pairs that exist in no paper. The numbers are ours, not quoted.
It changes the threat model	A defense can be the reason an attack works. That is a security claim, not a performance note.
It gives a design rule	Do not suppress the model's own memory. Held across two families and five implementations.
It predicted and then held	We stated the cause before the test. The measurement followed the predicted shape.

What is left

Finish the model comparison, add a third attack, and find an aggregation defense from another group.

Then the matrix is closed and the paper is written.